

Operating Systems

Assignment 3

Inter Process Communication

Sambit Sahoo (22277)

*My username is **sambit** (You can find it in all screenshots.)*

Q1. Write a C program using IPC – Shared Memory that performs the following two tasks using the producer-consumer paradigm:

1. **Producer (producer.c):** Ask the user for an integer and a string, and place them in shared memory.
2. **Consumer (consumer.c):** Read the data from shared memory, print the string entered by the producer, and check whether the input number is even or not.

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ vi a1_producer.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ gcc a1_producer.c -o a1_producer
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a1_producer
Enter a num:
22277
Enter a string:
sambitsahoo
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ vi a1_consumer.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ gcc a1_consumer.c -o a1_consumer
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a1_consumer
The word consumed is:
sambitsahoo
22277 is odd.
```

Producer Code

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
#include<stdio.h>           //for printf(), scanf()
#include<stdlib.h>          //for memory management (general utility)
#include<string.h>          //string manipulation func. (like strlen())
#include<fcntl.h>           //for O_CREAT, O_RDWR, etc.
#include<sys/shm.h>         //for shared memory operations
#include<sys/stat.h>        //file permission constants(like 0666, etc.)
#include<sys/mman.h>        //for PROT_WRITE
#include<unistd.h>          //for ftruncate

int main(){
    const int SIZE = 4096;
    const char *name = "SM1";

    int num;
    printf("Enter a num:\n");
    scanf("%d", &num);

    char *word = malloc(100 * sizeof(char));    // scanf store str in random mem -> prog may crash.
    printf("Enter a string:\n");
    scanf("%s", word);

    int shm_fd;
    void *ptr;

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE);
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* store integer */
    *((int*)ptr) = num;
    ptr += sizeof(int);

    sprintf(ptr, "%s\n", word);
    ptr += strlen(word);

    return 0;
}
a1_producer.c
```

Consumer Code

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<sys/shm.h>
#include<string.h>
#include<sys/mman.h>

int main(){
    const int SIZE = 4096;
    const char *name = "SM1";
    int shm_fd;
    void *ptr;
    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    int *a = (int*)ptr;
    ptr += sizeof(*a);

    char* b = (char *)ptr;
    printf("The word consumed is:\n");
    printf("%s", b);

    if (*a%2 != 0){
        printf("%d is odd.\n", *a);
    }
    else{
        printf("%d is even.\n", *a);
    }

    shm_unlink(name);
    return 0;
}

a1_consumer.c [ + ]
```

Producer Program Explanation

- Shared memory of size 4096 bytes is created using `shm_open()` with name `SM1`.
- The size of the shared memory is set using `ftruncate()`.
- `mmap()` maps the shared memory into the process address space.
- The program takes an integer and a string as input from the user.
- The integer is stored directly in shared memory using pointer casting.
- The pointer is moved forward by `sizeof(int)` to store the next data.
- The input string is written into shared memory using `sprintf()`.

Consumer Program Explanation

- Opens the shared memory object `SM1` using `shm_open()`.
- Maps the shared memory into the process address space using `mmap()` with read permission.

- Reads the first value from shared memory as an integer.
- Moves the pointer by sizeof(int) to access the stored string.
- Prints the consumed string from shared memory.
- Checks whether the consumed integer is even or odd using the modulus operator.
- Deletes the shared memory object using shm_unlink().

Q2. Write a C program using IPC – Shared Memory that performs the following tasks using the producer-consumer paradigm:

1. **Producer (producer.c):** Generate 100 random numbers and place them in shared memory.
2. **Consumer (consumer.c):** Read all 100 numbers from the shared memory, compute their cumulative sum, and then exit.

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ vi a2_producer.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ gcc a2_producer.c -o a2_producer
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a2_producer
100 random numbers written to shared memory.
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ vi a2_consumer.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ gcc a2_consumer.c -o a2_consumer
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a2_consumer
Sum = 49715
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a2_producer
100 random numbers written to shared memory.
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a2_consumer
Sum = 48086
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$
```

Producer Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/shm.h>
#include <time.h>

int main(){
    const int SIZE = 4096;
    const char *name = "SM2";

    int shm_fd;
    int *ptr;

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE);
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    srand(time(NULL)); // just to avoid the same seed

    for(int i = 0; i < 100; i++){
        ptr[i] = rand() % 1000; //ptr[i] is value stored at i (i.e it means *(ptr+i) = the number)
        //printf("%d\n", ptr[i]);
    }

    printf("100 random numbers written to shared memory.\n");

    return 0;
}
```

Consumer Code

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<fcntl.h>
#include<sys/stat.h>
#include<sys/shm.h>
#include<sys/mman.h>

int main(){
    const int SIZE = 4096;
    const char *name = "SM2";

    int shm_fd;
    int *ptr;
    int sum = 0;

    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    for (int i = 0; i < 100; i++){
        sum += ptr[i];
    }

    printf("Sum = %d\n", sum);

    shm_unlink(name);

    return 0;
}

a2_consumer.c 30,11-18 All
```

Producer Program Explanation

- Creates a shared memory object named SM2 using `shm_open()`.
- Sets the size of the shared memory to 4096 bytes using `ftruncate()`.
- Maps the shared memory into the process address space using `mmap()`.
- Seeds the random number generator using `srand(time(NULL))` to avoid repeating sequences.
- Generates 100 random integers using `rand()`.
- Stores the generated integers sequentially in shared memory using pointer indexing.
- Displays a confirmation message after writing the numbers.

Consumer Program Explanation

- Opens the shared memory object SM2 using `shm_open()` with read permission.
- Maps the shared memory into the process address space using `mmap()`.
- Reads 100 integers stored in shared memory using pointer indexing.
- Computes the cumulative sum of the numbers.
- Prints the computed sum to the terminal.
- Removes the shared memory object using `shm_unlink()` after computation.

Q3. Write a C program using IPC – Shared Memory that performs the following tasks:

1. Use shared memory to store an array of numbers.
2. The parent process writes 10 numbers into the shared memory.
3. Create three consumer processes such that:
 - (a) Consumer 1 computes the factorial of the first four numbers.
 - (b) Consumer 2 computes the factorial of the next two numbers.
 - (c) Consumer 3 computes the factorial of the remaining numbers.

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a3_producer
10 numbers stored in the array.
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a3_consumer1
Factorial of 6 is 720
Factorial of 0 is 1
Factorial of 2 is 2
Factorial of 7 is 5040
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a3_consumer2
Factorial of 5 is 120
Factorial of 1 is 1
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_3$ ./a3_consumer3
Factorial of 8 is 40320
Factorial of 6 is 720
Factorial of 4 is 24
Factorial of 0 is 1
```

Producer Code

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<sys/shm.h>
#include<string.h>
#include<sys/mman.h>
#include<time.h>

int main(){
    const int SIZE = 4096;
    const char *name = "SM3";
    int shm_fd;
    int *ptr;

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE);
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    srand(time(NULL));

    for(int i = 0; i < 10; i++){
        ptr[i] = rand() % 10;          //ptr[i] is value stored at i (i.e it means *(ptr+i) = the number)
        //printf("%d\n", ptr[i]);
    }

    printf("10 numbers stored in the array.\n");

    return 0;
}
```

a3_producer.c [+] 13,27-34 All

Consumer1 Code

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<sys/shm.h>
#include<string.h>
#include<sys/mman.h>

int fact(int n){
    if (n == 0 || n == 1){
        return 1;
    }
    else{
        return n * fact(n-1);
    }
}

int main(){
    const int SIZE = 4096;
    const char *name = "SM3";
    int shm_fd;
    int *ptr;
    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    for (int i = 0; i < 4; i++){
        printf("Factorial of %d is %d \n", ptr[i], fact(ptr[i]));
    }
    return 0;
}
a3_consumer1.c [+] 19,1 All
```

Consumer2 Code

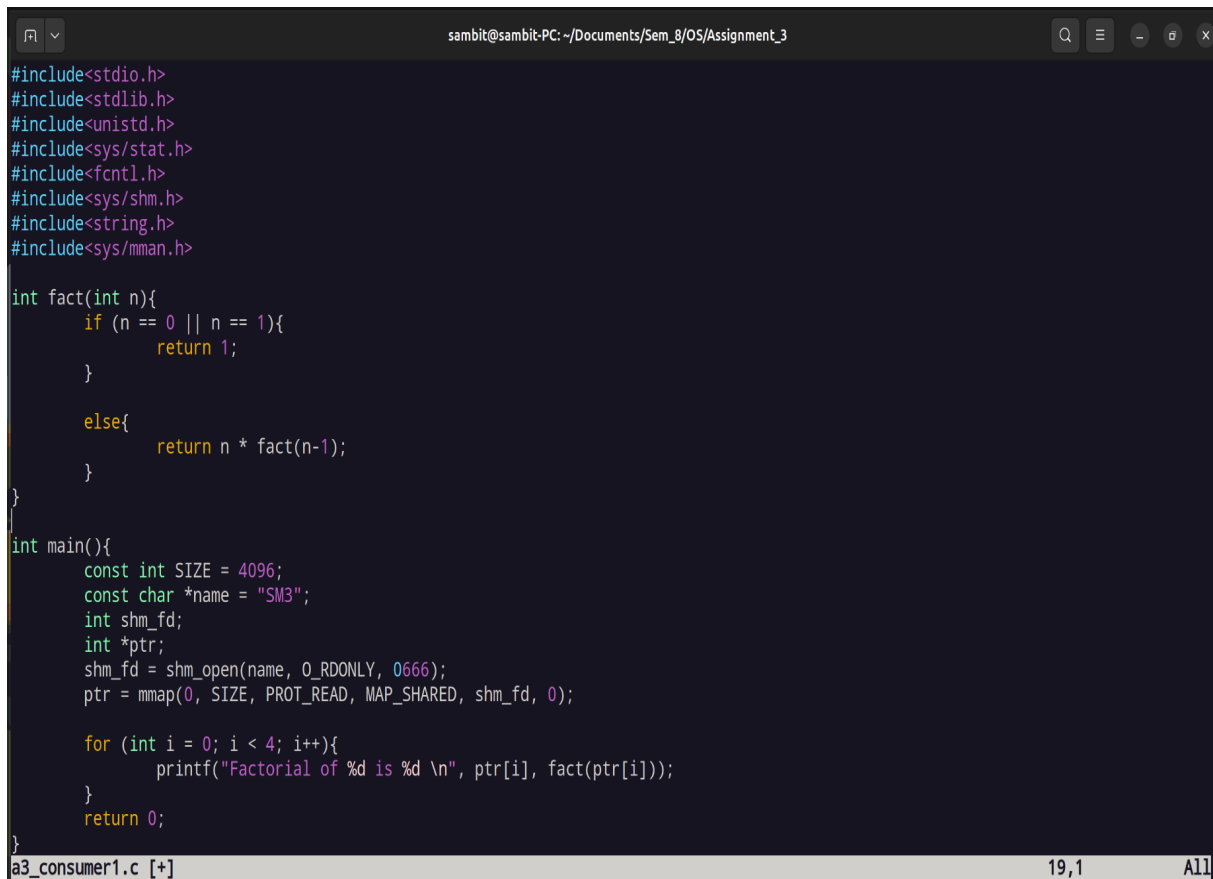
```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_3
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<sys/shm.h>
#include<string.h>
#include<sys/mman.h>

int fact(int n){
    if (n == 0 || n == 1){
        return 1;
    }
    else{
        return n * fact(n-1);
    }
}

int main(){
    const int SIZE = 4096;
    const char *name = "SM3";
    int shm_fd;
    int *ptr;
    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    for (int i = 0; i < 4; i++){
        printf("Factorial of %d is %d \n", ptr[i], fact(ptr[i]));
    }
    return 0;
}
a3_consumer1.c [+] 19,1 All
```

Consumer3 Code



```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<sys/shm.h>
#include<string.h>
#include<sys/mman.h>

int fact(int n){
    if (n == 0 || n == 1){
        return 1;
    }
    else{
        return n * fact(n-1);
    }
}

int main(){
    const int SIZE = 4096;
    const char *name = "SM3";
    int shm_fd;
    int *ptr;
    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    for (int i = 0; i < 4; i++){
        printf("Factorial of %d is %d \n", ptr[i], fact(ptr[i]));
    }
    return 0;
}
```

Producer Program Explanation

- Creates a shared memory object named SM3 using `shm_open()`.
- Sets the shared memory size to 4096 bytes using `ftruncate()`.
- Maps the shared memory into the process address space using `mmap()`.
- Seeds the random number generator with `srand(time(NULL))`.
- Generates 10 random integers using `rand()`.
- Stores the generated integers sequentially in shared memory using pointer indexing.
- Prints a confirmation message after storing the numbers.

Consumer Programs Explanation

- Each consumer process opens the shared memory object SM3 using `shm_open()`.
- The shared memory is mapped into the process address space using `mmap()` with read permission.
- The stored integers are accessed using pointer indexing.
- Each consumer computes the factorial of a subset of numbers using a recursive factorial function.

- Consumer1 processes the first four numbers (indices 0{3).
- Consumer2 processes the next two numbers (indices 4{5).
- Consumer3 processes the remaining numbers (indices 6{9).
- The computed factorial values are printed to the terminal.